7. Temporal Difference Method

과목 : 강화학습

담당 교수 : 박태형 교수님

학과 : 지능로봇공학과

학번 : 2023042028

이름 : 권택주

목차

과제 정의

- Temporal Difference Method
  - SARSA
  - Q-Learning

과제 풀이

- Temporal Difference Method
  - SARSA
  - Q-Learning

과제 결과

과제 정의

Temporal Difference Method

- 환경에 대한 사전 정보 없이 직접 경험을 통해 학습하는 Model Free 방식. 에피소드가 끝낼 때 까지 기다리지 않고 매 Time Step 마다 가치 함수를 업데이트 하는 Bootstrap 방식을 결합

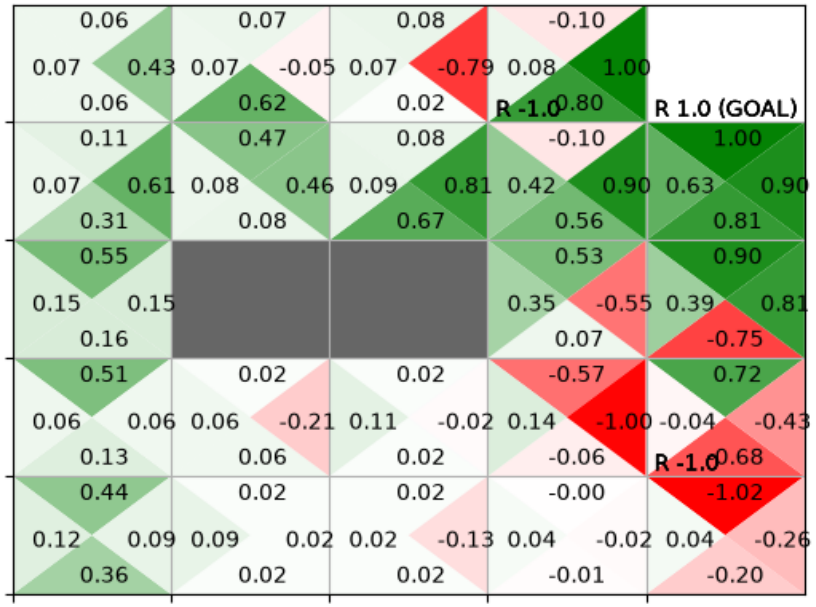
- **SARAS**
  - On Policy 정책. 현재 정책에 따라 실제로 선택할 다음 행동을 고려해 가치 함수를 업데이트.
  - $Q' = (S_t, A_t) = Q(S_t, A_t) + \alpha\{R_t + \gamma Q_\pi(S_{t+1}, A_{t+1}) - Q_\pi(S_t, A_t)\}$
- **Q-Learning**
  - Off Policy 정책. 실제 다음 행동과 무관하게, 다음 상태에서 가능한 모든 행동 중 가장 높은 가치를 갖는 행동을 무조건 선택한다 가정하고 업데이트
  - $Q' = (S_t, A_t) = Q(S_t, A_t) + \alpha\left\{R_t + \gamma \max_a Q\left(S_{t+1}, a\right) - Q(S_t, A_t)\right\}$

과제 풀이

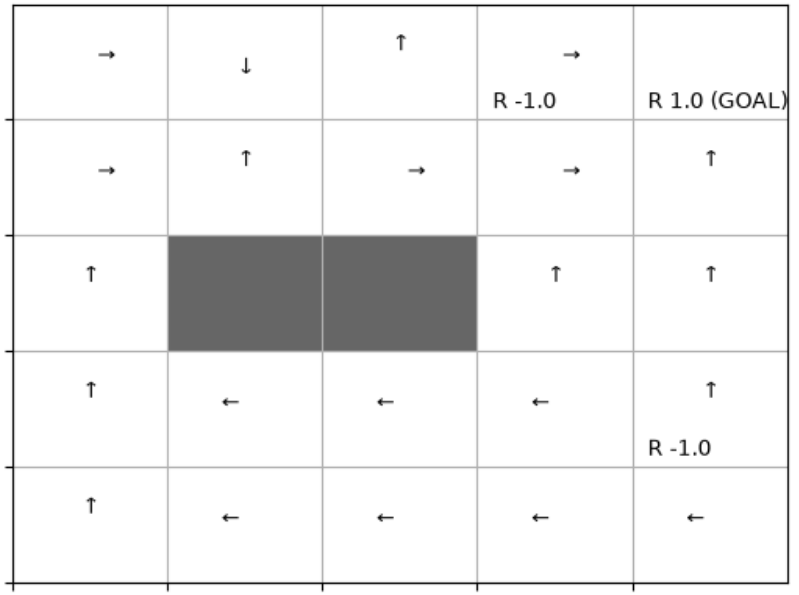
Temporal Difference Method – SARSA

기준 실험 α=0.8 · ε=0.1 · γ=0.9 · 10,000 episodes

Q-Value



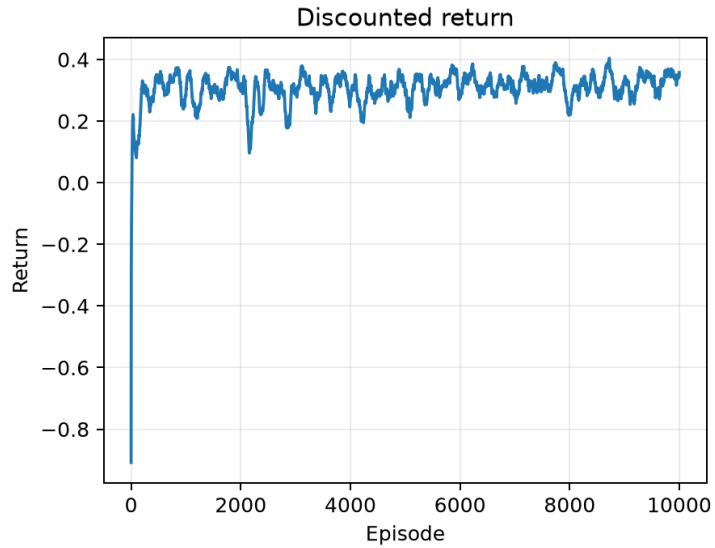
Greedy Policy

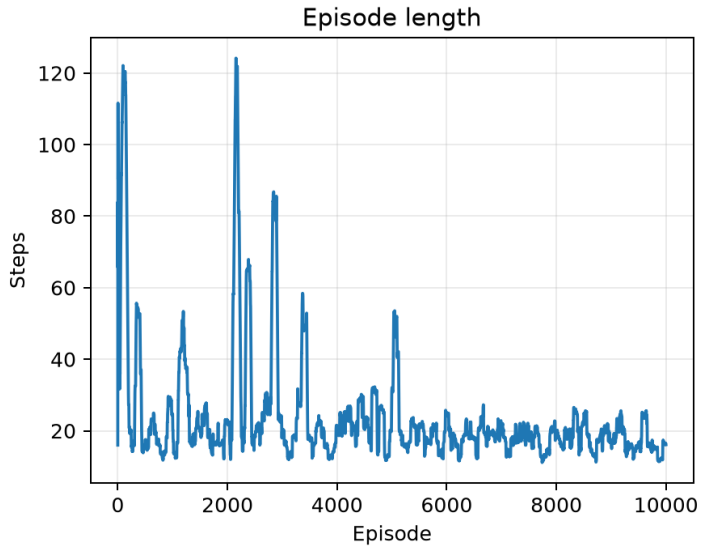


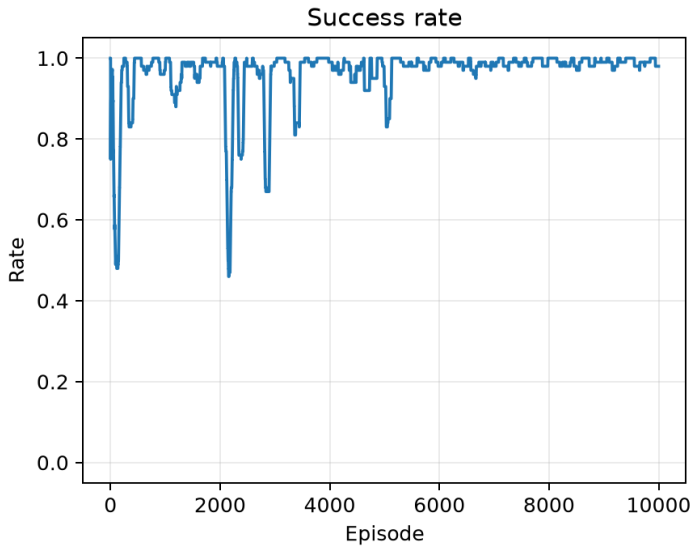
과제 풀이

Temporal Difference Method – SARSA | 학습 곡선

10,000회 학습 · 보상 / 스텝 / 종료 성공률 · 100-episode 이동평균







마지막 100회 평균보상

0.358

마지막 100회 평균 Step

16.11

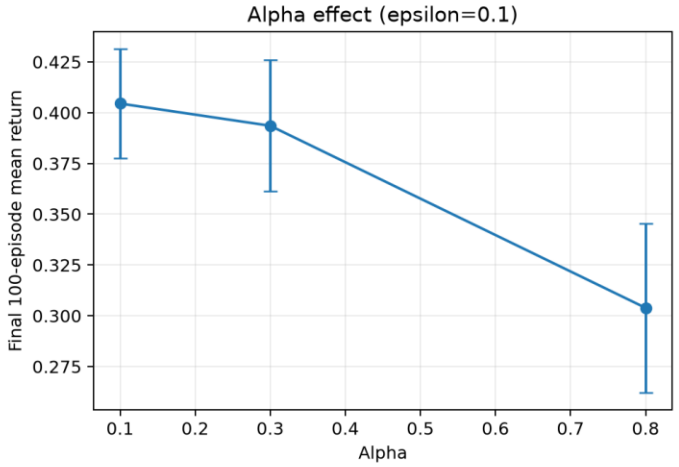
마지막 100회 종료 성공률

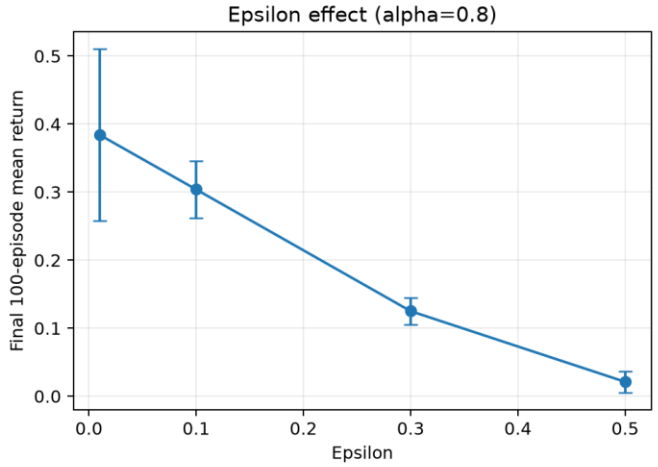
98%

과제 풀이

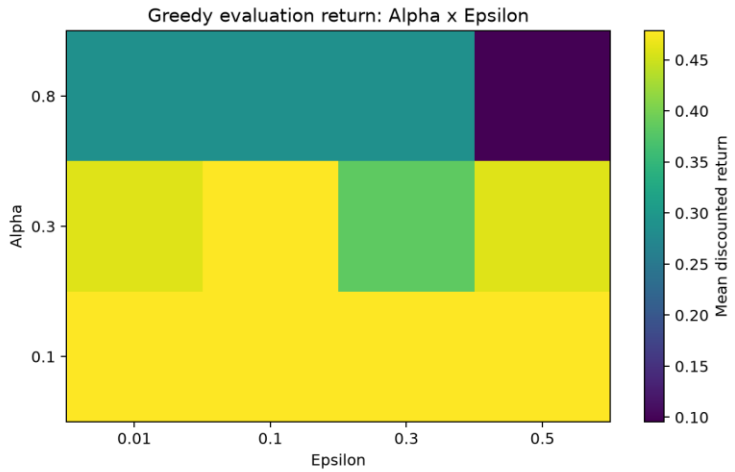
Temporal Difference Method – SARSA | Alpha, Epsilon

파라미터별 학습 보상





Greedy 평가 히트맵



과제 풀이

Temporal Difference Method – SARSA | Alpha, Epsilon

Alpha 변화에 따른 학습 보상

- **Alpha 0.1:** 가장 높은 평균 보상 0.405 기록. TD 오차를 작게 반영하여 Q-Value가 점진적으로 갱신됨.
- **Alpha 0.3:** 평균 보상 0.394 기록. 학습률 증가에 따라 새로운 경험의 반영 비율이 높아짐.
- **Alpha 0.8:** 평균 보상 0.304로 감소. 높은 학습률로 인해 Q-Value 변동성이 증가하고 실험 간 편차가 커짐.
- **분석:** Alpha 증가에 따라 평균 보상이 감소하는 경향을 확인. 탐색 행동까지 학습에 반영하는 On-Policy 특성과 관련됨.

Epsilon 변화에 따른 학습 보상

- **Epsilon 0.01:** 평균 보상 0.384 기록. 탐색 행동이 적어 높은 보상을 유지하지만 실험 간 편차가 크게 나타남.
- **Epsilon 0.1~0.3:** 평균 보상 0.304 → 0.125 감소. 탐색 확률 증가로 비최적 행동 및 우회 이동 발생 가능성 증가.
- **Epsilon 0.5:** 평균 보상 0.021 기록. 잦은 탐색으로 이동 효율이 감소함.
- **분석:** Epsilon 증가에 따라 학습 중 보상이 감소하며, SARSA는 실제 선택한 다음 행동을 TD Target에 반영함.

Alpha-Epsilon Greedy 평가 히트맵

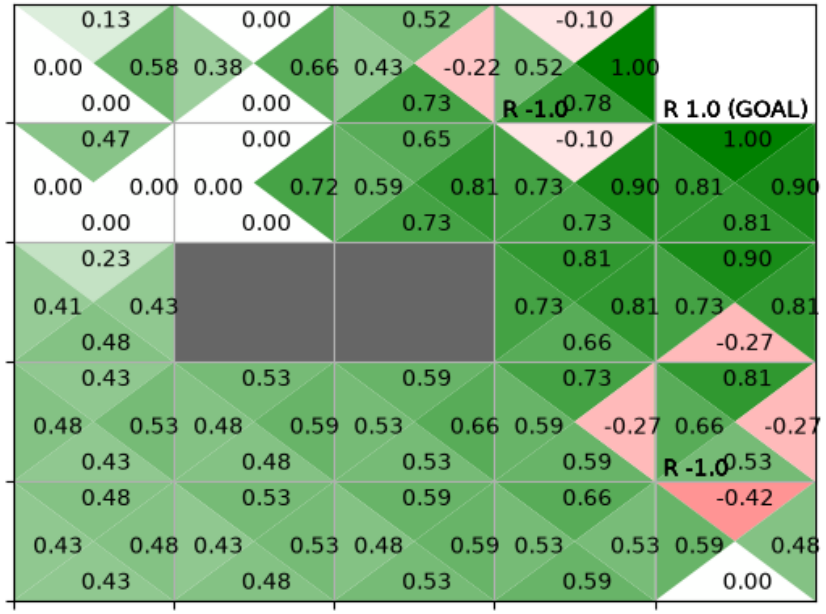
- **Alpha 0.1:** 모든 Epsilon 조건에서 평가 보상 0.478 기록. 안정적인 최종 정책 형성.
- **Alpha 0.3:** 일부 조합에서 평가 보상 0.383~0.460으로 감소. 파라미터에 따른 정책 성능 차이 발생.
- **Alpha 0.8:** Epsilon 0.5에서 평가 보상 0.096, 목표 도달 성공률 20% 기록.
- **분석:** 높은 Alpha 조건에서 최종 정책의 성능 저하와 실험 간 편차가 확인됨.

과제 풀이

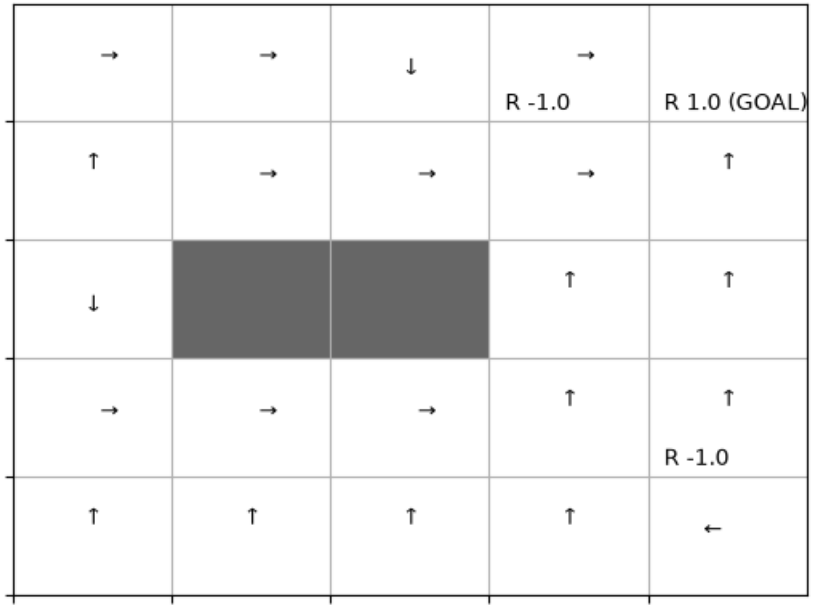
Temporal Difference Method – Q-Learning

기준 실험 α=0.8 · ε=0.1 · γ=0.9 · 10,000 episodes

Q-Value



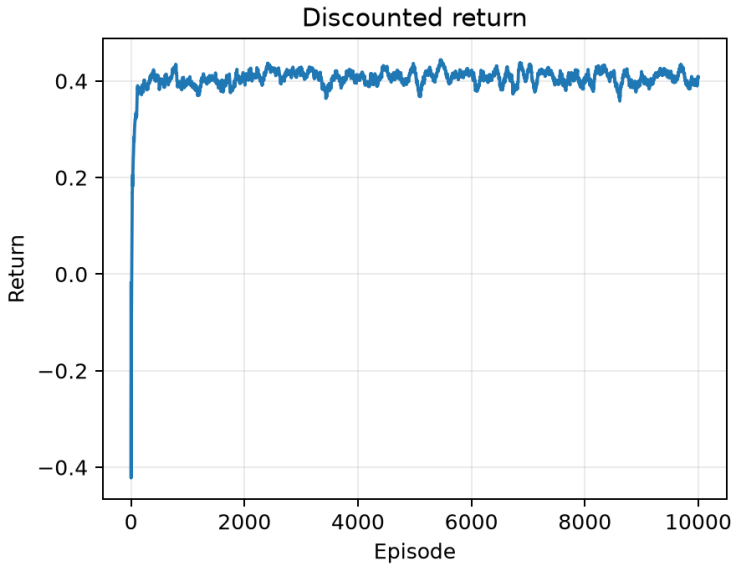
Greedy Policy

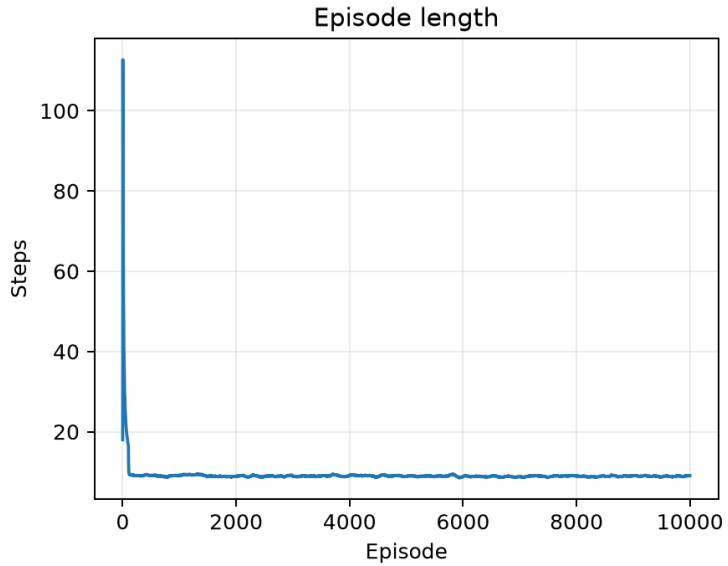


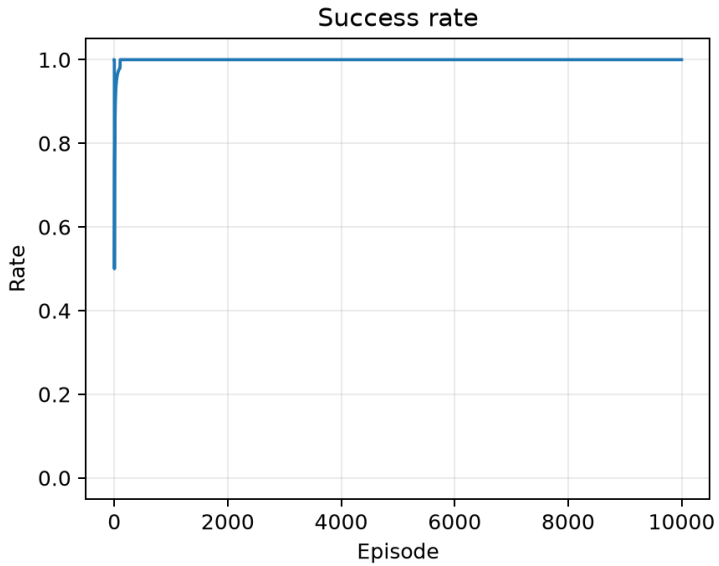
과제 풀이

Temporal Difference Method – Q-Learning | 학습 곡선

10,000회 학습 · 보상 / 스텝 / 종료 성공률 · 100-episode 이동평균







마지막 100회 평균보상

0.407

마지막 100회 평균 Step

9.02

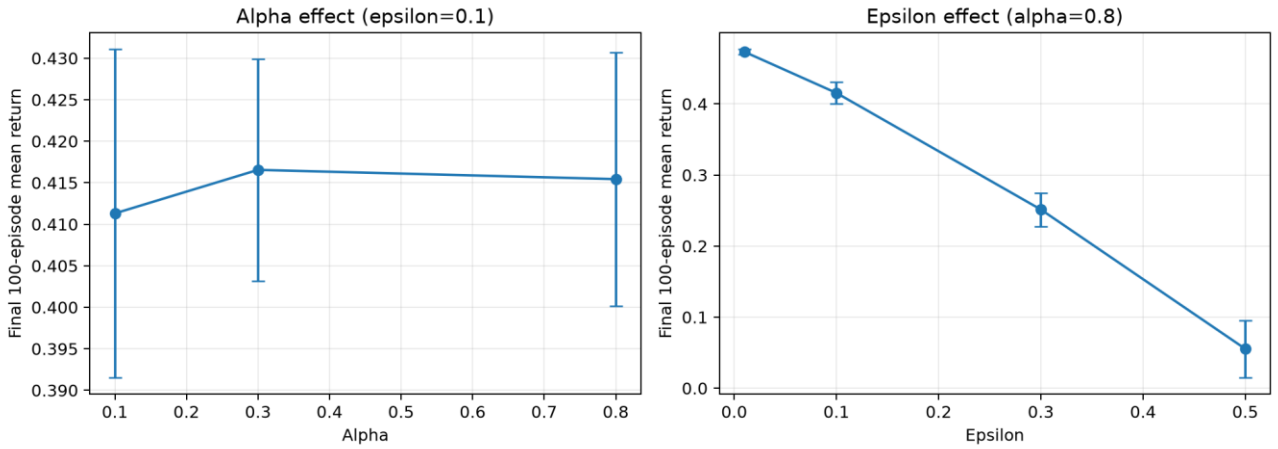
마지막 100회 종료 성공률

100%

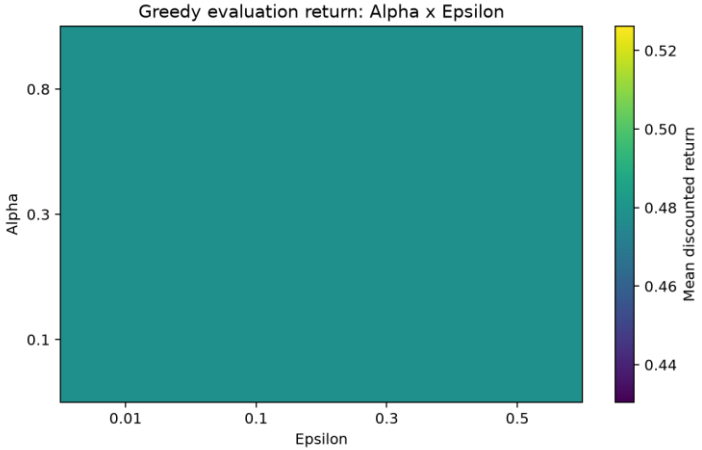
과제 풀이

Temporal Difference Method – Q-Learning | Alpha, Epsilon

파라미터별 학습 보상



Greedy 평가 히트맵



과제 풀이

Temporal Difference Method – Q-Learning | Alpha, Epsilon

Alpha 변화에 따른 학습 보상

- **Alpha 0.1:** 평균 보상 0.411 기록. 작은 학습률로 Q-Value를 점진적으로 갱신함.
- **Alpha 0.3:** 가장 높은 평균 보상 0.417 기록. TD 오차 반영 비율 증가에도 안정적인 학습 성능 유지.
- **Alpha 0.8:** 평균 보상 0.415 기록. 높은 학습률에서도 큰 성능 저하가 발생하지 않음.
- **분석:** Alpha 변화에도 평균 보상 0.411~0.417 유지. 최대 Q-Value를 활용하는 Off-Policy 학습 특성과 관련됨.

Epsilon 변화에 따른 학습 보상

- **Epsilon 0.01:** 가장 높은 평균 보상 0.473 기록. 탐색 비율이 낮아 효율적인 이동 경로를 자주 선택함.
- **Epsilon 0.1~0.3:** 평균 보상 0.415 → 0.251 감소. 탐색 확률 증가에 따른 비최적 행동 발생.
- **Epsilon 0.5:** 평균 보상 0.055 기록. 높은 탐색 확률로 학습 중 이동 효율 감소.
- **분석:** Epsilon 증가에 따라 학습 보상이 감소함. 다음 상태의 최대 Q-Value를 사용하여 최적 행동 가치를 학습함.

Alpha–Epsilon Greedy 평가 히트맵

- **전체 파라미터 조합:** Alpha 0.1~0.8, Epsilon 0.01~0.5에서 평가 보상 0.478 기록.
- **목표 도달:** 총 60회 실험 모두 8스텝으로 목표에 도달하며 성공률 100% 기록.
- **학습 안정성:** 학습률과 탐색 확률 변화에도 최종 Greedy 평가 성능이 동일하게 나타남.
- **분석:** 모든 파라미터 조합에서 높은 최종 정책 성능을 확보. 해당 GridWorld 환경에서 안정적인 학습 결과를 확인함.

과제 결과

1. SARAS

- Alpha가 0.1에서 0.8로 증가함에 따라 평균 학습 보상이 0.405에서 0.304로 감소함. Epsilon 증가에 따라 학습 보상이 감소하며, 높은 Alpha, Epsilon 조합에서는 Greedy 정책의 목표 도달 실패 발생.
- 이는 실제로 선택한 다음 행동의 Q-Value를 업데이트에 반영하는 On-Policy 특성과 관련있는 것으로 분석됨.

2. Q-Learning

- Alpha 변화에도 평균 학습 보상이 0.411~0.417로 일정하게 유지. Epsilon 증가에 따라 학습 중 보상은 감소하였지만, 모든 파라미터 조합에서 Greedy 평가 보상 0.478을 기록하며 8 Step으로 목표 도달.
- 다음 상태의 최대 Q-Value를 사용하는 Off-Policy 업데이트 특성이 이번 실험의 안정적 최종 평가 성능에 기여한 것으로 해석됨

sarsa_one_cell

October 15, 2026

[2]:
```python
# GridWorld TD control experiment
# Requires the original textbook's common/gridworld.py and common/utils.py.
import os
import sys
from collections import defaultdict
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

# Keep the existing course-project folder structure.
sys.path.append(os.path.abspath('..'))
from common.gridworld import GridWorld
from common.utils import greedy_probs

GAMMA = 0.9
BASELINE_EPISODES = 10000
SWEEP_EPISODES = 3000
MAX_STEPS = 200
ALPHAS = [0.1, 0.3, 0.8]
EPSILONS = [0.01, 0.1, 0.3, 0.5]
SEEDS = range(5)

class SarsaAgent:
    def __init__(self, alpha=0.8, epsilon=0.1, gamma=GAMMA):
        self.alpha = alpha
        self.epsilon = epsilon
        self.gamma = gamma
        self.action_size = 4
        self.Q = defaultdict(float)
        self.pi = defaultdict(
            lambda: {a: 1 / self.action_size for a in range(self.action_size)}
        )

    def get_action(self, state):
        probs = self.pi[state]
```

```
        return int(np.random.choice(list(probs), p=list(probs.values())))

    def update(self, state, action, reward, next_state, next_action, done):
        next_q = 0.0 if done else self.Q[next_state, next_action]
        target = reward + self.gamma * next_q
        self.Q[state, action] += self.alpha * (target - self.Q[state, action])
        self.pi[state] = greedy_probs(self.Q, state, self.epsilon)

def train(alpha=0.8, epsilon=0.1, episodes=BASELINE_EPISODES,
          seed=42, max_steps=MAX_STEPS):
    np.random.seed(seed)
    env = GridWorld()
    agent = SarsaAgent(alpha=alpha, epsilon=epsilon)
    returns = np.zeros(episodes)
    lengths = np.zeros(episodes, dtype=int)
    successes = np.zeros(episodes, dtype=int)

    for ep in range(episodes):
        state = env.reset()
        action = agent.get_action(state)

        for t in range(max_steps):
            next_state, reward, done = env.step(action)
            next_action = None if done else agent.get_action(next_state)
            agent.update(state, action, reward, next_state, next_action, done)
            returns[ep] += (agent.gamma ** t) * reward
            lengths[ep] = t + 1

            if done:
                successes[ep] = 1
                break
            state, action = next_state, next_action

    return agent, returns, lengths, successes

def evaluate_greedy(agent, max_steps=MAX_STEPS):
    """One deterministic rollout from GridWorld.reset()."""
    env = GridWorld()
    state = env.reset()
    total_return = 0.0

    for t in range(max_steps):
        action = int(np.argmax([
            agent.Q[state, a] for a in range(agent.action_size)
        ]))
        state, reward, done = env.step(action)
        total_return += (agent.gamma ** t) * reward
```

```
        if done:
            return 1, total_return, t + 1

    return 0, total_return, np.nan  # A timed-out episode is not successful.


def moving_average(values, window=100):
    return pd.Series(values).rolling(window, min_periods=1).mean()


def plot_learning_curves(returns, lengths, successes, output_dir):
    fig, axs = plt.subplots(1, 3, figsize=(15, 4))
    for ax, arr, title, ylabel in zip(
        axs,
        [returns, lengths, successes],
        ['Discounted return', 'Episode length', 'Success rate'],
        ['Return', 'Steps', 'Rate']
    ):
        ax.plot(moving_average(arr), linewidth=1.5)
        ax.set(title=title, xlabel='Episode', ylabel=ylabel)
        ax.grid(alpha=0.25)
    axs[2].set_ylim(-0.05, 1.05)
    fig.tight_layout()
    fig.savefig(output_dir / 'learning_curves.png', dpi=180)
    plt.show()
    plt.close(fig)


def parameter_sweep(algorithm, output_dir):
    records = []
    total = len(ALPHAS) * len(EPSILONS) * len(SEEDS)
    finished = 0

    for alpha in ALPHAS:
        for epsilon in EPSILONS:
            for seed in SEEDS:
                agent, returns, lengths, successes = train(
                    alpha=alpha, epsilon=epsilon,
                    episodes=SWEEP_EPISODES, seed=seed
                )
                eval_success, eval_return, eval_steps = evaluate_greedy(agent)
                records.append({
                    'algorithm': algorithm,
                    'alpha': alpha,
                    'epsilon': epsilon,
                    'seed': seed,
                    'train_return_last100': returns[-100:].mean(),
```

```
                    'train_success_last100': successes[-100:].mean(),
                    'train_steps_last100': lengths[-100:].mean(),
                    'eval_success': eval_success,
                    'eval_return': eval_return,
                    'eval_steps': eval_steps,
                })
                finished += 1
                if finished % len(SEEDS) == 0:
                    print(f'[{algorithm}] {finished}/{total} experiments')

    results = pd.DataFrame(records)
    results.to_csv(output_dir / 'sweep_raw.csv', index=False)
    summary = results.groupby(['alpha', 'epsilon']).agg(
        train_return_mean=('train_return_last100', 'mean'),
        train_return_std=('train_return_last100', 'std'),
        train_success_mean=('train_success_last100', 'mean'),
        eval_return_mean=('eval_return', 'mean'),
        eval_return_std=('eval_return', 'std'),
        eval_success_mean=('eval_success', 'mean'),
        eval_success_std=('eval_success', 'std'),
        eval_steps_mean=('eval_steps', 'mean'),
    ).reset_index()
    summary.to_csv(output_dir / 'sweep_summary.csv', index=False)
    print(summary.round(3).to_string(index=False))
    return results, summary


def plot_parameter_results(summary, output_dir):
    matrix = summary.pivot(index='alpha', columns='epsilon',
                           values='eval_return_mean')
    fig, ax = plt.subplots(figsize=(7, 4.5))
    image = ax.imshow(matrix.to_numpy(), aspect='auto', origin='lower')
    ax.set_xticks(range(len(matrix.columns)), matrix.columns)
    ax.set_yticks(range(len(matrix.index)), matrix.index)
    ax.set(xlabel='Epsilon', ylabel='Alpha',
           title='Greedy evaluation return: Alpha x Epsilon')
    fig.colorbar(image, ax=ax, label='Mean discounted return')
    fig.tight_layout()
    fig.savefig(output_dir / 'alpha_epsilon_heatmap.png', dpi=180)
    plt.show()
    plt.close(fig)

    # Controlled 1D comparisons: fix epsilon (alpha test), fix alpha (epsilon
    ↪test).
    fixed_epsilon, fixed_alpha = 0.1, 0.8
    a = summary[np.isclose(summary['epsilon'], fixed_epsilon)]
    e = summary[np.isclose(summary['alpha'], fixed_alpha)]
```

```
    fig, axs = plt.subplots(1, 2, figsize=(11, 4))
    axs[0].errorbar(a['alpha'], a['train_return_mean'],
                    yerr=a['train_return_std'], marker='o', capsize=4)
    axs[0].set(title=f'Alpha effect (epsilon={fixed_epsilon})',
               xlabel='Alpha', ylabel='Final 100-episode mean return')
    axs[1].errorbar(e['epsilon'], e['train_return_mean'],
                    yerr=e['train_return_std'], marker='o', capsize=4)
    axs[1].set(title=f'Epsilon effect (alpha={fixed_alpha})',
               xlabel='Epsilon', ylabel='Final 100-episode mean return')
    for ax in axs:
        ax.grid(alpha=0.25)
    fig.tight_layout()
    fig.savefig(output_dir / 'alpha_epsilon_effects.png', dpi=180)
    plt.show()
    plt.close(fig)

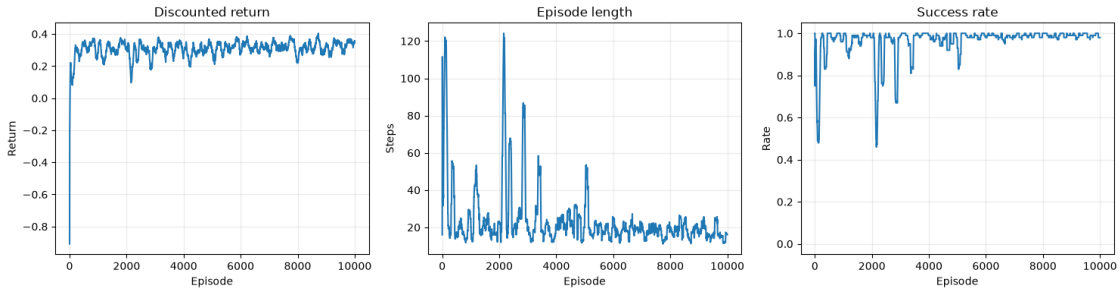
def main():
    output_dir = Path('td_results') / 'sarsa'
    output_dir.mkdir(parents=True, exist_ok=True)

    agent, returns, lengths, successes = train(seed=42)
    print('SARSA baseline (last 100 episodes):')
    print(f'  mean discounted return: {returns[-100:].mean():.3f}')
    print(f'  mean episode length: {lengths[-100:].mean():.2f}')
    print(f'  success rate: {successes[-100:].mean():.3f}')
    print(f'  greedy evaluation (success, return, steps): '
          f'{evaluate_greedy(agent)}')
    plot_learning_curves(returns, lengths, successes, output_dir)
    _, summary = parameter_sweep('SARSA', output_dir)
    plot_parameter_results(summary, output_dir)

    # Original assignment output: state-action Q-values and greedy policy.
    GridWorld().render_q(agent.Q)

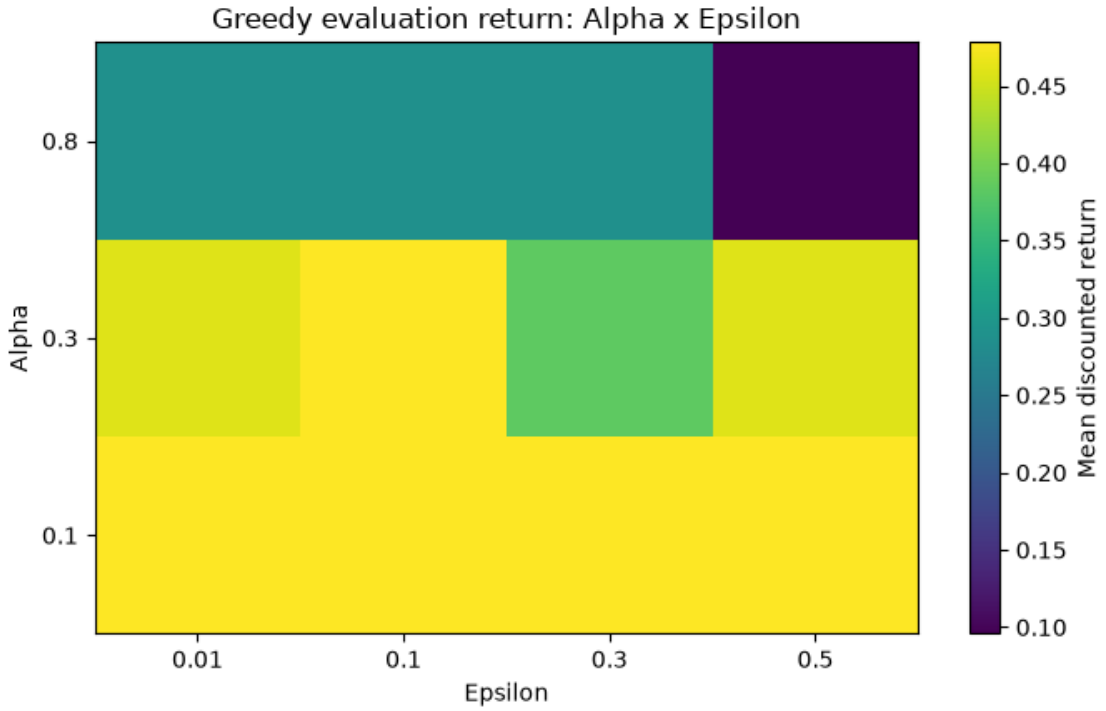

# Execute all experiments and render all results from this single cell.
main()
```

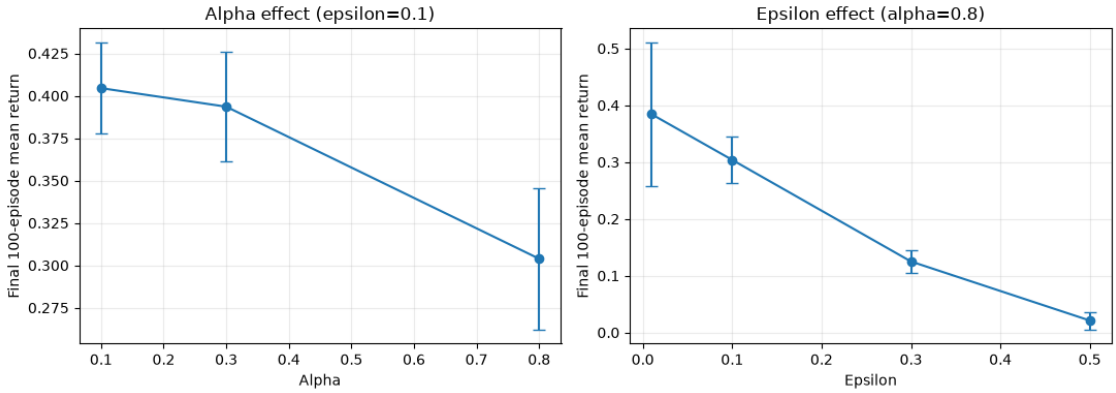
```
SARSA baseline (last 100 episodes):
  mean discounted return: 0.358
  mean episode length: 16.11
  success rate: 0.980
  greedy evaluation (success, return, steps): (0, 0.0, nan)
```

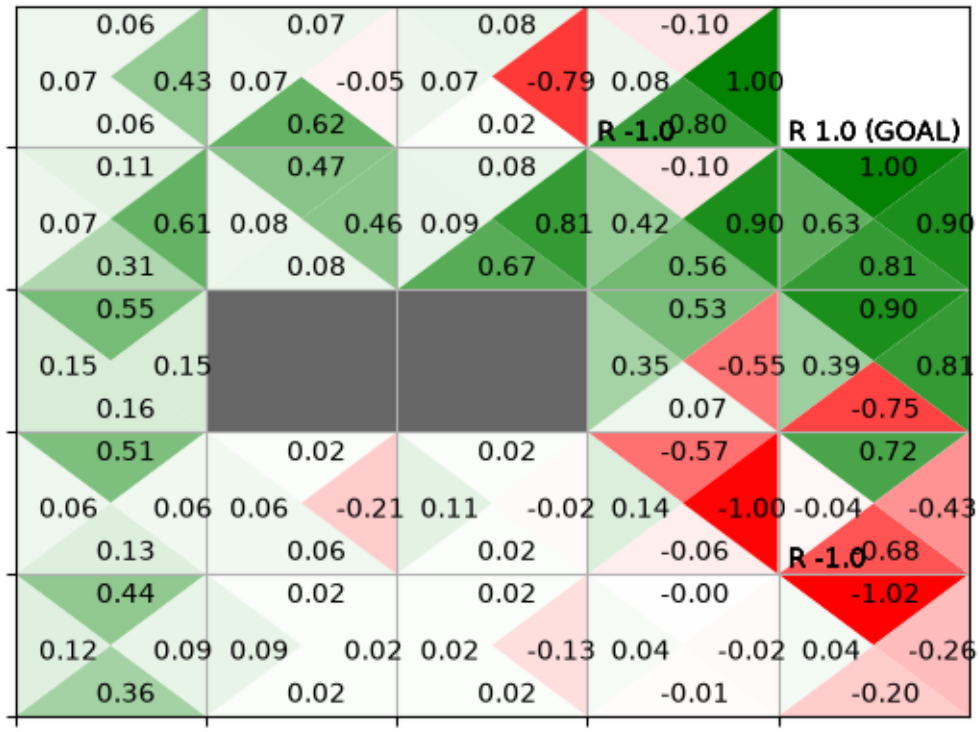


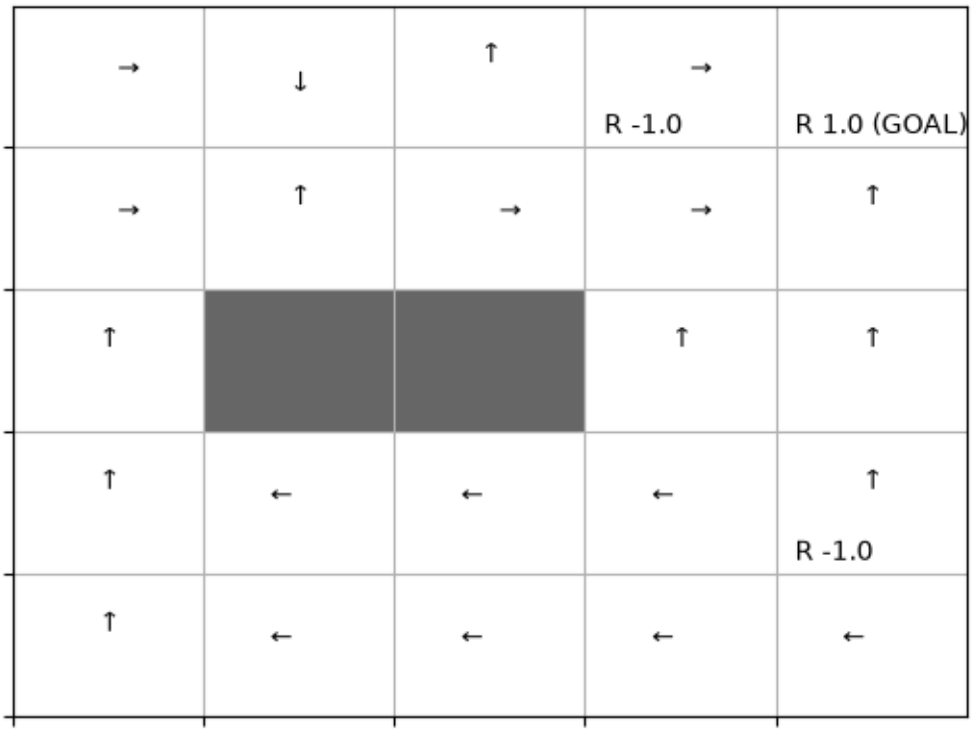

```
[SARSA] 5/60 experiments
[SARSA] 10/60 experiments
[SARSA] 15/60 experiments
[SARSA] 20/60 experiments
[SARSA] 25/60 experiments
[SARSA] 30/60 experiments
[SARSA] 35/60 experiments
[SARSA] 40/60 experiments
[SARSA] 45/60 experiments
[SARSA] 50/60 experiments
[SARSA] 55/60 experiments
[SARSA] 60/60 experiments
 alpha  epsilon  train_return_mean  train_return_std  train_success_mean
eval_return_mean  eval_return_std  eval_success_mean  eval_success_std
eval_steps_mean
   0.1     0.01              0.473             0.003                0.998
0.478            0.000              1.0             0.000              8.0
   0.1     0.10              0.405             0.027                1.000
0.478            0.000              1.0             0.000              8.0
   0.1     0.30              0.273             0.026                1.000
0.478            0.000              1.0             0.000              8.0
   0.1     0.50              0.129             0.028                1.000
0.478            0.000              1.0             0.000              8.0
   0.3     0.01              0.439             0.041                1.000
0.460            0.041              1.0             0.000              8.4
   0.3     0.10              0.394             0.032                0.992
0.478            0.000              1.0             0.000              8.0
   0.3     0.30              0.241             0.015                0.996
0.383            0.214              0.8             0.447              8.0
   0.3     0.50              0.106             0.041                0.998
0.460            0.041              1.0             0.000              8.4
   0.8     0.01              0.384             0.126                0.844
0.287            0.262              0.6             0.548              8.0
   0.8     0.10              0.304             0.042                0.966
0.287            0.262              0.6             0.548              8.0
   0.8     0.30              0.125             0.020                0.950
```

```
0.287                0.262                0.6                0.548                8.0
   0.8       0.50                 0.021                0.016                0.956
0.096                0.214                0.2                0.447                8.0
```






0.06 0.07 0.08 -0.10
0.07 0.43 0.07 -0.05 0.07 -0.79 0.08 1.00
0.06 0.62 0.02 R -1.0 0.80 R 1.0 (GOAL)
0.11 0.47 0.08 -0.10 1.00
0.07 0.61 0.08 0.46 0.09 0.81 0.42 0.90 0.63 0.90
0.31 0.08 0.67 0.56 0.81
0.55 0.53 0.90
0.15 0.15 0.35 -0.55 0.39 0.81
0.16 0.07 -0.75
0.51 0.02 0.02 -0.57 0.72
0.06 0.06 0.06 -0.21 0.11 -0.02 0.14 -1.00 -0.04 -0.43
0.13 0.06 0.02 -0.06 R -1.0 0.68
0.44 0.02 0.02 -0.00 -1.02
0.12 0.09 0.09 0.02 0.02 -0.13 0.04 -0.02 0.04 -0.26
0.36 0.02 0.02 -0.01 -0.20


→ ↓ ↑ → R -1.0 R 1.0 (GOAL)
→ ↑ → → ↑
↑ ↑ ↑
↑ ← ← ← ↑ R -1.0
↑ ← ← ← ←

q_learning_one_cell

October 15, 2026

[1]:
```python
# GridWorld TD control experiment
# Requires the original textbook's common/gridworld.py and common/utils.py.
import os
import sys
from collections import defaultdict
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

# Keep the existing course-project folder structure.
sys.path.append(os.path.abspath('..'))
from common.gridworld import GridWorld
from common.utils import greedy_probs

GAMMA = 0.9
BASELINE_EPISODES = 10000
SWEEP_EPISODES = 3000
MAX_STEPS = 200
ALPHAS = [0.1, 0.3, 0.8]
EPSILONS = [0.01, 0.1, 0.3, 0.5]
SEEDS = range(5)

class QLearningAgent:
    def __init__(self, alpha=0.8, epsilon=0.1, gamma=GAMMA):
        self.alpha = alpha
        self.epsilon = epsilon
        self.gamma = gamma
        self.action_size = 4
        self.Q = defaultdict(float)
        initial_probs = lambda: {
            a: 1 / self.action_size for a in range(self.action_size)
        }
        self.pi = defaultdict(initial_probs)  # Target (greedy) policy
        self.b = defaultdict(initial_probs)   # Behavior (epsilon-greedy) policy
```

```
    def get_action(self, state):
        probs = self.b[state]
        return int(np.random.choice(list(probs), p=list(probs.values())))

    def update(self, state, action, reward, next_state, done):
        next_q_max = (
            0.0 if done else max(self.Q[next_state, a]
                                 for a in range(self.action_size))
        )
        target = reward + self.gamma * next_q_max
        self.Q[state, action] += self.alpha * (target - self.Q[state, action])
        self.pi[state] = greedy_probs(self.Q, state, epsilon=0)
        self.b[state] = greedy_probs(self.Q, state, self.epsilon)

def train(alpha=0.8, epsilon=0.1, episodes=BASELINE_EPISODES,
          seed=42, max_steps=MAX_STEPS):
    np.random.seed(seed)
    env = GridWorld()
    agent = QLearningAgent(alpha=alpha, epsilon=epsilon)
    returns = np.zeros(episodes)
    lengths = np.zeros(episodes, dtype=int)
    successes = np.zeros(episodes, dtype=int)

    for ep in range(episodes):
        state = env.reset()

        for t in range(max_steps):
            action = agent.get_action(state)
            next_state, reward, done = env.step(action)
            agent.update(state, action, reward, next_state, done)
            returns[ep] += (agent.gamma ** t) * reward
            lengths[ep] = t + 1

            if done:
                successes[ep] = 1
                break
            state = next_state

    return agent, returns, lengths, successes

def evaluate_greedy(agent, max_steps=MAX_STEPS):
    """One deterministic rollout from GridWorld.reset()."""
    env = GridWorld()
    state = env.reset()
    total_return = 0.0

    for t in range(max_steps):
```

```
        action = int(np.argmax([
            agent.Q[state, a] for a in range(agent.action_size)
        ]))
        state, reward, done = env.step(action)
        total_return += (agent.gamma ** t) * reward
        if done:
            return 1, total_return, t + 1

    return 0, total_return, np.nan  # A timed-out episode is not successful.


def moving_average(values, window=100):
    return pd.Series(values).rolling(window, min_periods=1).mean()


def plot_learning_curves(returns, lengths, successes, output_dir):
    fig, axs = plt.subplots(1, 3, figsize=(15, 4))
    for ax, arr, title, ylabel in zip(
        axs,
        [returns, lengths, successes],
        ['Discounted return', 'Episode length', 'Success rate'],
        ['Return', 'Steps', 'Rate']
    ):
        ax.plot(moving_average(arr), linewidth=1.5)
        ax.set(title=title, xlabel='Episode', ylabel=ylabel)
        ax.grid(alpha=0.25)
    axs[2].set_ylim(-0.05, 1.05)
    fig.tight_layout()
    fig.savefig(output_dir / 'learning_curves.png', dpi=180)
    plt.show()
    plt.close(fig)


def parameter_sweep(algorithm, output_dir):
    records = []
    total = len(ALPHAS) * len(EPSILONS) * len(SEEDS)
    finished = 0

    for alpha in ALPHAS:
        for epsilon in EPSILONS:
            for seed in SEEDS:
                agent, returns, lengths, successes = train(
                    alpha=alpha, epsilon=epsilon,
                    episodes=SWEEP_EPISODES, seed=seed
                )
                eval_success, eval_return, eval_steps = evaluate_greedy(agent)
                records.append({
```

```
                    'algorithm': algorithm,
                    'alpha': alpha,
                    'epsilon': epsilon,
                    'seed': seed,
                    'train_return_last100': returns[-100:].mean(),
                    'train_success_last100': successes[-100:].mean(),
                    'train_steps_last100': lengths[-100:].mean(),
                    'eval_success': eval_success,
                    'eval_return': eval_return,
                    'eval_steps': eval_steps,
                })
                finished += 1
                if finished % len(SEEDS) == 0:
                    print(f'[{algorithm}] {finished}/{total} experiments')

    results = pd.DataFrame(records)
    results.to_csv(output_dir / 'sweep_raw.csv', index=False)
    summary = results.groupby(['alpha', 'epsilon']).agg(
        train_return_mean=('train_return_last100', 'mean'),
        train_return_std=('train_return_last100', 'std'),
        train_success_mean=('train_success_last100', 'mean'),
        eval_return_mean=('eval_return', 'mean'),
        eval_return_std=('eval_return', 'std'),
        eval_success_mean=('eval_success', 'mean'),
        eval_success_std=('eval_success', 'std'),
        eval_steps_mean=('eval_steps', 'mean'),
    ).reset_index()
    summary.to_csv(output_dir / 'sweep_summary.csv', index=False)
    print(summary.round(3).to_string(index=False))
    return results, summary


def plot_parameter_results(summary, output_dir):
    matrix = summary.pivot(index='alpha', columns='epsilon',
                           values='eval_return_mean')
    fig, ax = plt.subplots(figsize=(7, 4.5))
    image = ax.imshow(matrix.to_numpy(), aspect='auto', origin='lower')
    ax.set_xticks(range(len(matrix.columns)), matrix.columns)
    ax.set_yticks(range(len(matrix.index)), matrix.index)
    ax.set(xlabel='Epsilon', ylabel='Alpha',
           title='Greedy evaluation return: Alpha x Epsilon')
    fig.colorbar(image, ax=ax, label='Mean discounted return')
    fig.tight_layout()
    fig.savefig(output_dir / 'alpha_epsilon_heatmap.png', dpi=180)
    plt.show()
    plt.close(fig)
```

```
    # Controlled 1D comparisons: fix epsilon (alpha test), fix alpha (epsilon␣
 ↪test).
    fixed_epsilon, fixed_alpha = 0.1, 0.8
    a = summary[np.isclose(summary['epsilon'], fixed_epsilon)]
    e = summary[np.isclose(summary['alpha'], fixed_alpha)]
    fig, axs = plt.subplots(1, 2, figsize=(11, 4))
    axs[0].errorbar(a['alpha'], a['train_return_mean'],
                    yerr=a['train_return_std'], marker='o', capsize=4)
    axs[0].set(title=f'Alpha effect (epsilon={fixed_epsilon})',
               xlabel='Alpha', ylabel='Final 100-episode mean return')
    axs[1].errorbar(e['epsilon'], e['train_return_mean'],
                    yerr=e['train_return_std'], marker='o', capsize=4)
    axs[1].set(title=f'Epsilon effect (alpha={fixed_alpha})',
               xlabel='Epsilon', ylabel='Final 100-episode mean return')
    for ax in axs:
        ax.grid(alpha=0.25)
    fig.tight_layout()
    fig.savefig(output_dir / 'alpha_epsilon_effects.png', dpi=180)
    plt.show()
    plt.close(fig)

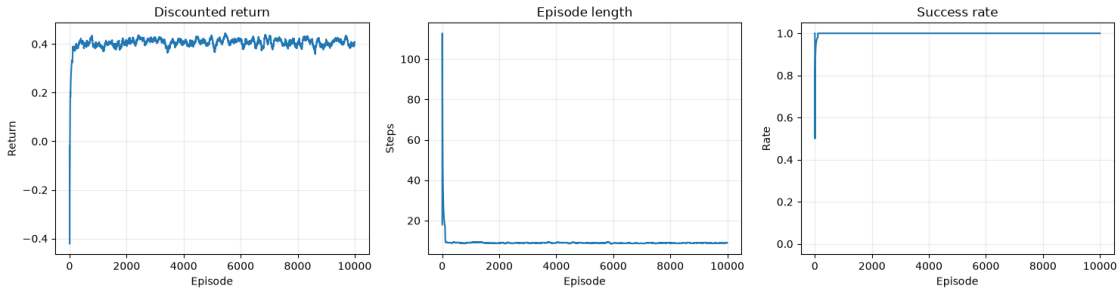
def main():
    output_dir = Path('td_results') / 'q_learning'
    output_dir.mkdir(parents=True, exist_ok=True)

    agent, returns, lengths, successes = train(seed=42)
    print('Q-Learning baseline (last 100 episodes):')
    print(f'  mean discounted return: {returns[-100:].mean():.3f}')
    print(f'  mean episode length: {lengths[-100:].mean():.2f}')
    print(f'  success rate: {successes[-100:].mean():.3f}')
    print(f'  greedy evaluation (success, return, steps): '
          f'{evaluate_greedy(agent)}')
    plot_learning_curves(returns, lengths, successes, output_dir)
    _, summary = parameter_sweep('Q-Learning', output_dir)
    plot_parameter_results(summary, output_dir)

    # Original assignment output: state-action Q-values and greedy policy.
    GridWorld().render_q(agent.Q)

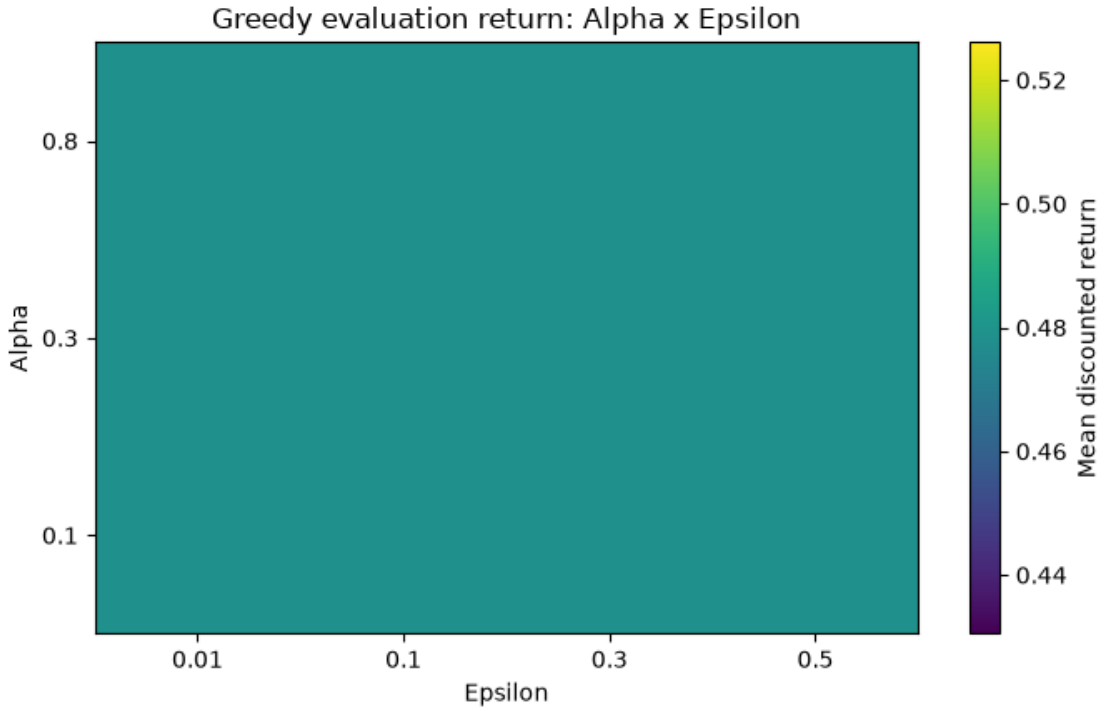

# Execute all experiments and render all results from this single cell.
main()
```

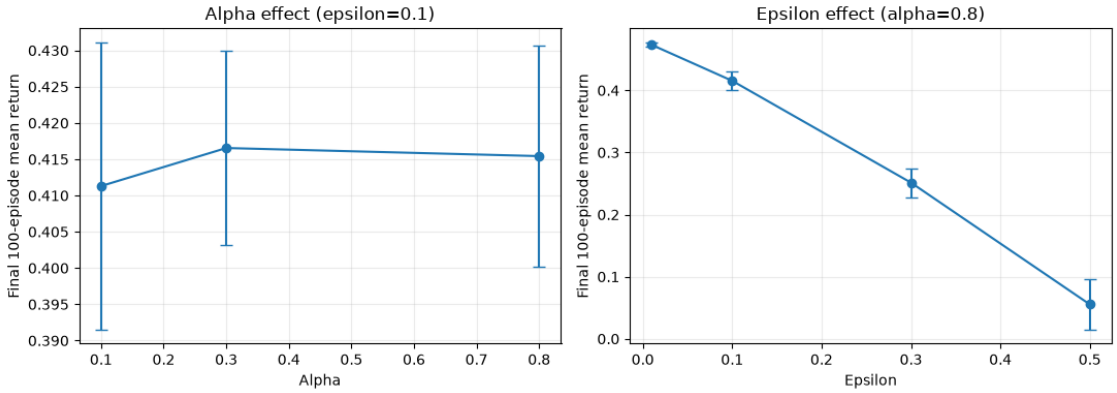
```
Q-Learning baseline (last 100 episodes):
  mean discounted return: 0.407
  mean episode length: 9.02
  success rate: 1.000
  greedy evaluation (success, return, steps): (1, 0.4782969000000001, 8)
```

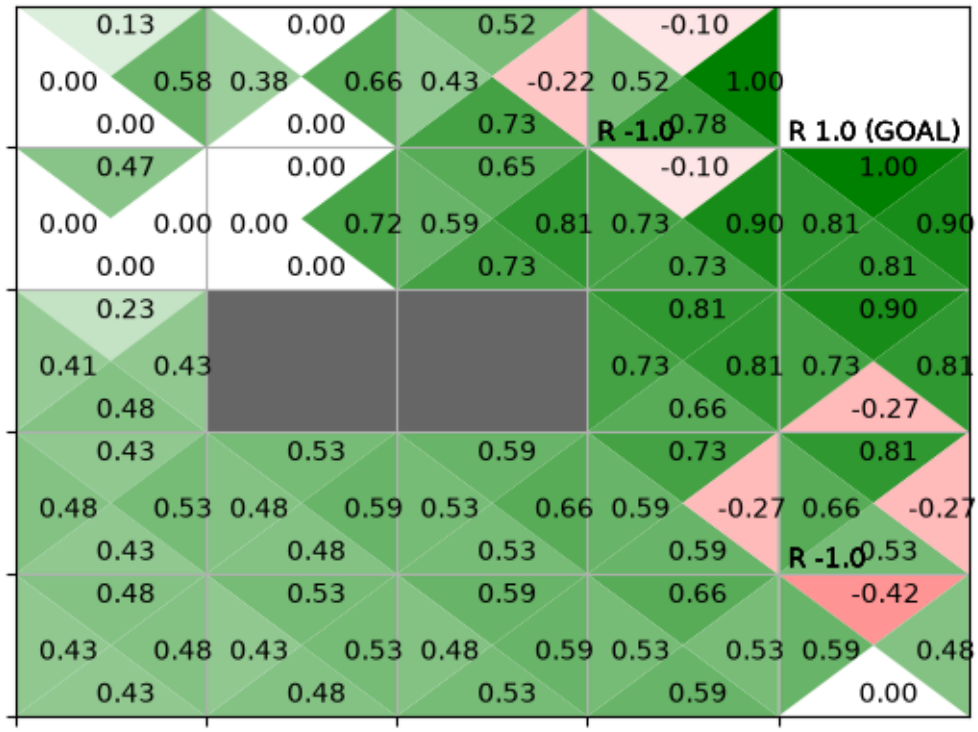


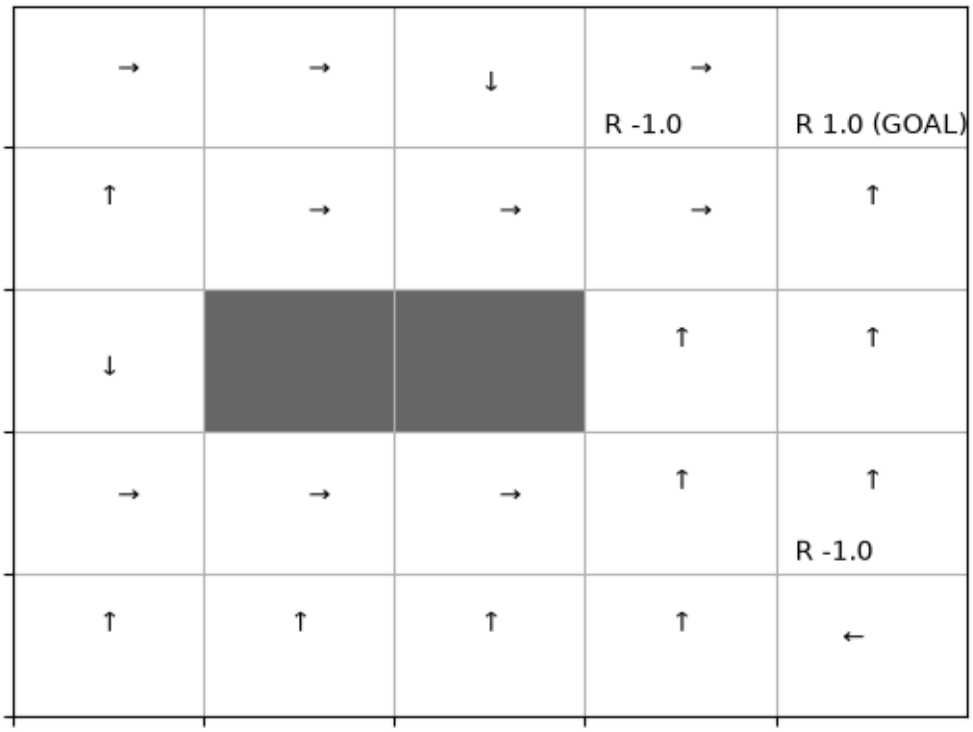

```
[Q-Learning] 5/60 experiments
[Q-Learning] 10/60 experiments
[Q-Learning] 15/60 experiments
[Q-Learning] 20/60 experiments
[Q-Learning] 25/60 experiments
[Q-Learning] 30/60 experiments
[Q-Learning] 35/60 experiments
[Q-Learning] 40/60 experiments
[Q-Learning] 45/60 experiments
[Q-Learning] 50/60 experiments
[Q-Learning] 55/60 experiments
[Q-Learning] 60/60 experiments
 alpha  epsilon  train_return_mean  train_return_std  train_success_mean
eval_return_mean  eval_return_std  eval_success_mean  eval_success_std
eval_steps_mean
   0.1     0.01              0.473             0.003                  1.0
0.478              0.0                1.0               0.0               8.0
   0.1     0.10              0.411             0.020                  1.0
0.478              0.0                1.0               0.0               8.0
   0.1     0.30              0.214             0.029                  1.0
0.478              0.0                1.0               0.0               8.0
   0.1     0.50              0.071             0.047                  1.0
0.478              0.0                1.0               0.0               8.0
   0.3     0.01              0.473             0.003                  1.0
0.478              0.0                1.0               0.0               8.0
   0.3     0.10              0.417             0.013                  1.0
0.478              0.0                1.0               0.0               8.0
   0.3     0.30              0.250             0.024                  1.0
0.478              0.0                1.0               0.0               8.0
   0.3     0.50              0.056             0.027                  1.0
0.478              0.0                1.0               0.0               8.0
   0.8     0.01              0.473             0.003                  1.0
0.478              0.0                1.0               0.0               8.0
   0.8     0.10              0.415             0.015                  1.0
0.478              0.0                1.0               0.0               8.0
   0.8     0.30              0.251             0.023                  1.0
```

```
0.478                0.0                1.0                0.0                8.0
   0.8      0.50                0.055                0.040                1.0
0.478                0.0                1.0                0.0                8.0
```






0.13
0.00
0.52
-0.10
0.00
0.58
0.38
0.66
0.43
-0.22
0.52
1.00
0.00
0.00
0.73
R -1.0
0.78
R 1.0 (GOAL)
0.47
0.00
0.65
-0.10
1.00
0.00
0.00
0.00
0.72
0.59
0.81
0.73
0.90
0.81
0.90
0.00
0.00
0.73
0.73
0.81
0.23
0.81
0.90
0.41
0.43
0.73
0.81
0.73
0.81
0.48
0.66
-0.27
0.43
0.53
0.59
0.73
0.81
0.48
0.53
0.48
0.59
0.53
0.66
0.59
-0.27
0.66
-0.27
0.43
0.48
0.53
0.59
R -1.0
0.53
0.48
0.53
0.59
0.66
-0.42
0.43
0.48
0.43
0.53
0.48
0.59
0.53
0.53
0.59
0.48
0.43
0.48
0.53
0.59
0.00


R -1.0
R 1.0 (GOAL)
R -1.0